<<route>>
ContractRoutes
+GET /contracts (AuthGuard, PermissionGuard)
+GET /contracts/:id (AuthGuard, PermissionGuard)
+POST /contracts (AuthGuard, PermissionGuard)
+PATCH /contracts/:id (AuthGuard, PermissionGuard)
+PATCH /contracts/:id/sign (AuthGuard, PermissionGuard)
+PATCH /contracts/:id/complete (AuthGuard, PermissionGuard)
+PATCH /contracts/:id/cancel (AuthGuard, PermissionGuard)
maps to
ContractController
-contractService: ContractService
+getContractList(userId, roleName, query: GetContractListQueryDTO)
+getContractDetail(userId, roleName, contractId: number)
+createContract(userId, body: CreateContractBodyDTO)
+updateContract(userId, contractId: number, body: UpdateContractBodyDTO)
+signContract(userId, contractId: number)
+completeContract(userId, contractId: number)
+cancelContract(userId, contractId: number)
AuthGuard
-tokenService: TokenService
-prisma: PrismaService
-reflector: Reflector
+canActivate(context: ExecutionContext): Promise<boolean>
-extractTokenFromHeader(request: Request): string | undefined
PermissionGuard
-reflector: Reflector
-sharedPermissionRepository: SharedPermissionRepository
+canActivate(context: ExecutionContext): Promise<boolean>
calls
<<dto>>
CreateContractBodyDTO
+proposalId: number
+totalAmount: number
+terms?: string
+expiresAt?: string
<<dto>>
UpdateContractBodyDTO
+totalAmount?: number
+terms?: string
+expiresAt?: string
<<dto>>
GetContractListQueryDTO
+page?: number = 1
+limit?: number = 10
+status?: ContractStatus
+proposalId?: number
<<dto>>
GetContractListResponseDTO
+data: ContractDetailType[]
+totalItems: number
+totalPages: number
+page: number
+limit: number
<<type>>
ContractDetailType
+client: { id, email, profile }
+freelancer: { id, email, profile }
+job: { id, title, slug }
ContractService
-contractRepository: ContractRepository
+createContract(clientId: number, body: CreateContractBodyType)
+updateContract(clientId: number, contractId: number, body: UpdateContractBodyType)
+signContract(userId: number, contractId: number)
+completeContract(clientId: number, contractId: number)
+cancelContract(userId: number, contractId: number)
+getContractList(userId: number, roleName: string, query: GetContractListQueryType)
+getContractDetail(userId: number, roleName: string, contractId: number)
calls
ContractRepository
-prisma: PrismaService
+findProposalWithJob(proposalId: number)
+findContractByProposalId(proposalId: number)
+findContractById(contractId: number)
+createContract(data: CreateContractBodyType, clientId, freelancerId, jobId)
+updateContract(contractId: number, data: UpdateContractBodyType)
+signContract(contractId: number, role: string)
+completeContract(contractId: number, jobId: number)
+cancelContract(contractId: number)
+getContractList(query: GetContractListQueryType, filter: object)
+getContractDetail(contractId: number)
maps to
maps to
maps to
<<schema>>
Contract
+id: number
+jobId: number
+proposalId: number
+clientId: number
+freelancerId: number
+terms: string | null
+totalAmount: Decimal
+status: ContractStatus
+signedByClient: boolean
+signedByFreelancer: boolean
+platformFee: Decimal
+platformFeeStatus: PlatformFeeStatus
+platformFeePaidAt: Date | null
+createdAt: Date
+signedAt: Date | null
+completedAt: Date | null
+expiresAt: Date | null
+deletedAt: Date | null
uses
uses
<<enum>>
ContractStatus
+PENDING_SIGN
+ACTIVE
+COMPLETED
+DISPUTED
+CANCELLED
<<enum>>
PlatformFeeStatus
+UNPAID
+PAID
+WAIVED
1
<<schema>>
Job
+id: number
+clientId: number
+title: string
+slug: string
+status: JobStatus
+createdAt: Date
1..*
<<schema>>
Proposal
+id: number
+jobId: number
+freelancerId: number
+bidAmount: Decimal
+status: ProposalStatus
+createdAt: Date
1
<<schema>>
User
+id: number
+email: string
+roleId: number
+isBanned: boolean
+deletedAt: Date | null